

## Esercizi di preparazione - tipologie [4]-[8]

---

### Esercizi

#### [4] | 2 punti | Cosa stampa il seguente codice?

```
def doppio(n):  
    n = n * 2  
    return n  
  
x = 3  
y = doppio(x)  
print(x, y)
```

- 3 3
- 3 6
- 6 6
- 6 3

---

#### [5] | 6 punti | Considera il seguente codice sorgente e rispondi alle domande che seguono

Il seguente programma vuole costruire la lista che, per ogni elemento di `lista`, conta quanti elementi **uguali a lui** lo **precedono** nella lista.

```
lista = [1, 2, 1, 3, 1]  
  
contatori = []  
i = 0  
while i < len(lista):  
    contatore = 0  
    j = 0  
    while j < i:  
        if lista[j] == lista[i]:  
            contatore = contatore + 1  
    contatori.append(contatore)  
    i = i + 1  
  
print(contatori)
```

- Il programma termina senza errori?
  - L'output è quello atteso? Se c'è un errore logico, spiegallo e scrivi il codice corretto.
-

**[6] | 4 punti | Completa la tabella di tracciamento dello stato per il seguente frammento di codice**

```
def raddoppia(lista):
    i = 0
    while i < len(lista):
        lista[i] = lista[i] * 2
        i = i + 1

valori = [1, 3, 5]
raddoppia(valori)
print(valori)
```

| Passo | Codice sorgente | Stato - globale | Stato - funzione | Output |
|-------|-----------------|-----------------|------------------|--------|
|       |                 |                 |                  |        |
|       |                 |                 |                  |        |
|       |                 |                 |                  |        |
|       |                 |                 |                  |        |
|       |                 |                 |                  |        |
|       |                 |                 |                  |        |
|       |                 |                 |                  |        |
|       |                 |                 |                  |        |

**[7] | 6 punti | Scrivi una funzione void chiamata `moltiplica_per(lista, k)`**

La funzione riceve una lista di interi e un intero `k`, e moltiplica ogni elemento della lista per `k`, **modificando la lista originale** (nessun valore di ritorno).

Esempio: dopo `moltiplica_per([1, 2, 3, 4], 3)`, la lista `[1, 2, 3, 4]` diventa `[3, 6, 9, 12]`.

**[7] | 6 punti | Scrivi una funzione pura chiamata `filtra_pari(lista)`**

La funzione riceve una lista di interi e restituisce una nuova lista contenente solo i numeri **pari**, senza modificare la lista di partenza.

Esempio: Input: `[1, 2, 3, 4, 5, 6]`

Output:

```
[2, 4, 6]
```

**[8] | 10 punti | Scrivi una funzione pura chiamata `alterna(lista, k)`**

La funzione riceve una lista di interi e un intero `k`, e restituisce una nuova lista contenente solo gli elementi agli indici multipli di `k` (cioè gli indici `0, k, 2k, ...`), senza modificare la lista di partenza.

Esempio: `alterna([10, 20, 30, 40, 50, 60], 2) → [10, 30, 50]`

Poi scrivi un programma che legge un intero `k` dall'input controllando che sia maggiore di zero (validazione dell'input), legge 6 numeri interi e li salva in una lista, e stampa il risultato di `alterna` chiamata con `k` e poi con `k + 1`.

Esempio con `k = 2` e lista `[10, 20, 30, 40, 50, 60]`:

Output:

```
[10, 30, 50]
[10, 40]
```

#### **[8] | 10 punti | Scrivi una funzione pura chiamata `inserisci_dopo_pari(lista, k)`**

La funzione riceve una lista di interi e un intero `k`, e restituisce una nuova lista in cui dopo ogni numero pari viene inserito `k`, senza modificare la lista di partenza.

Esempio: `inserisci_dopo_pari([3, 4, 9, 2, 8, 7], 0) → [3, 4, 0, 9, 2, 0, 8, 0, 7]`

Poi scrivi un programma che legge un intero `k` dall'input, legge numeri interi fino a `0` e li salva in una lista, e stampa il risultato di `inserisci_dopo_pari`.